

Linked lists

C programming · Structures and dynamic memory

ISTIC · University of Rennes 2026–2027



1/84

Kahoot corrections



Dynamic memory allocation · q1-q10

2/84

Kahoot (alloc) q1: pointer errors



```
/* X */ m = malloc(5); m = NULL;
/* Y */ free(n); n->value = 5;
/* Z */ char *p; *p = 'a';
```

- Using dangling pointers
- Using uninitialised pointers
- Lost memory

Match X, Y and Z to the problems.

- A. X-1, Y-3, Z-2
- B. X-3, Y-2, Z-1
- C. X-2, Y-1, Z-3
- D. **X-3, Y-1, Z-2**

- X leaks if allocation succeeds; its address is lost.
- Y uses a freed object; Z has no initialised address.

3/84

Kahoot (alloc) q2: **free(NULL)**



```
#include <stdio.h>
int main() {
    int *p = (int *)malloc(sizeof(int));
    p = NULL;
    free(p);
}
```

- free(NULL)** does nothing (CM, slide 291).
- D assumes allocation succeeds. No alias is saved here.
- Missing `<stdlib.h>` is a separate C17 error.

What is the problem with this code?

- A. Compiler error: **free()** can't be applied on NULL pointer
- B. Dangling pointer
- C. The program may crash as **free()** is called for NULL pointer.
- D. **Memory leak**

4/84

Kahoot (alloc) q2: another reference



```
int *p = malloc(sizeof(int));
if (p == NULL) return EXIT_FAILURE;
int *q = p;
p = NULL;
free(p);
free(q);
```

- q kept the allocated address before p changed.
- free(q)** releases the block. **No leak here.**

5/84

Kahoot (alloc) q3: return value of **free()**



```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int *ptr1, *ptr2;
    ptr1 = malloc(4);
    *ptr1 = 10;
    *ptr2 = free(ptr1);
    printf("%d\n", *ptr2);
    return 0;
}
```

What is the output?

- A. 10
- B. The address stored in ptr1
- C. The address stored in ptr2
- D. **Error**

- free()** returns **void**: no value to assign.
- ptr2 is also uninitialised.

6/84

Kahoot (alloc) q4: a local address



```
#include <stdio.h>
int *fun() {
    int x = 5;
    return &x;
}
int main() {
    int *p = fun();
    printf("%d", *p);
    return 0;
}
```

What is the problem with this code?

- A. Memory leak
 - B. **Dangling pointer**
 - C. There is no problem
 - D. Compilation error
- x stops existing when `fun()` returns.
 - Dereferencing `p` has **undefined behaviour**.

7/84

Kahoot (alloc) q5: one block



```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int r = 3, c = 4;
    int *arr = (int *)malloc(
        r * c * sizeof(int));
    int i, j, count = 0;
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            ??? = ++count;
    /* ... */
    return 0;
}
```

Which expression replaces ????

- A. ***(&arr + i*c + j)**
- B. `arr[i][j]`
- C. `arr + i*c + j`
- D. ***(&arr[i] + &arr[j])**

- Skip `i` rows of `c` integers, then `j` integers.
- Also written `arr[i*c + j]`.

8/84

Kahoot (alloc) q6: separate rows



```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int r = 3, c = 4, i, j, count;
    int *arr[r];
    for (i = 0; i < r; i++)
        arr[i] = (int *)malloc(
            c * sizeof(int));
    count = 0;
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            ??? = ++count;
    /* ... */
    return 0;
}
```

Which expression replaces ????

- A. ***(&arr + i*c + j)**
- B. `*(&arr + i*c) + j`
- C. `*(&arr + i*c + j)`
- D. ***(&arr[i] + j)**

- `*(&arr + i)` selects row `i`; `+j` selects column `j`.
- Also written `arr[i][j]` or `*(&arr[i])[j]`.

9/84

Kahoot (alloc) q7: freeing the grid



```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int r = 3, c = 4, i, j, count;
    int **arr = (int **)malloc(
        r * sizeof(int *));
    for (i = 0; i < r; i++)
        arr[i] = (int *)malloc(
            c * sizeof(int));
    count = 0;
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            arr[i][j] = ++count;
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            printf("%d ", arr[i][j]);
    free(arr);
    return 0;
}
```

Is this code correct?

- A. Yes
- B. **No**
- C. I do not know

- `free(arr)` releases only the pointer table.
- On success, **the three rows leak**. Free them first:

```
for (i = 0; i < r; i++)
    free(arr[i]);
free(arr);
```

10/84

Kahoot (alloc) q8: initialisation



```
struct Point {
    int x = 0;
    int y = 0;
};
```

What is the output? (C17)

- A. `x=0, y=0`
- B. **Compilation error**
- C. Nothing of the mentioned

- A C structure definition cannot initialise its members.
- Initialise an **object** after defining its type.

11/84

Kahoot (alloc) q9: structure copy



```
#include <stdio.h>
struct student { char *name; };
struct student s;
struct student fun(void) {
    s.name = "newton";
    printf("%s\n", s.name);
    s.name = "alan";
    return s;
}
void main() {
    struct student m = fun();
    printf("%s\n", m.name);
    m.name = "turing";
    printf("%s\n", s.name);
}
```

What is the output?

- A. **newton · alan · alan**
- B. `alan · newton · alan`
- C. `newton · alan · turing`
- D. Compilation error

- Assigning `m.name` does not change `s.name`.
- A assumes `void main()` is accepted; strict compilation can reject it. Use `int main(void)`.

12/84

Kahoot (alloc) q9: the caller



```
struct student m = fun();
printf("%s\n", m.name);
m.name = "turing";
printf("%s\n", s.name);
```

Pointer value	After <code>fun()</code>	After assignment
<code>m.name</code> points to	"alan"	"turing"
<code>s.name</code> points to	"alan"	"alan"

- Returning `s` copies its pointer member into `m`.
- The last `printf()` reads `s.name`, still `alan`.

13/84

Kahoot (alloc) q10: member access



```
#include <stdio.h>
struct point { int x; int y; };
void foo(struct point *);
int main() {
    struct point p1 = {1, 2};
    foo(&p1);
}
void foo(struct point *p) {
    printf("%d\n", *p.x++);
}
```

What is the output?

A. **Compile time error**

B. Segmentation fault

C. 2

D. 1

- `*p.x++` . binds before `*`.
- `p` is a pointer. `p.x++` or `(*p).x++` prints 1, then sets `x` to 2.

14/84

Struct reminder



Each list node will hold a song ID and a pointer.

A structure groups them together.

15/84

Struct: type and object



```
struct Point {
    int x;
    int y;
};
struct Point origin = {0, 0};
origin.x = 3;
```

- `struct Point` describes the type; `origin` is an object.
- `origin.x` accesses that object's `x` member.

16/84

Typedef: a type alias



```
typedef struct Point point_t;
struct Point a = {1, 2};
point_t b = {1, 2};
```

- `struct Point` and `point_t` name the **same type**.
- `typedef` creates no object and allocates no heap memory.

17/84

1. A playlist that changes



A listener requests a song next.

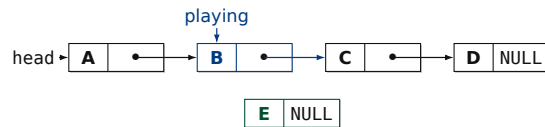
Can we insert it without shifting the rest of the playlist?

18/84

Play E immediately after B



- **B is playing.** A listener requests **E next.**



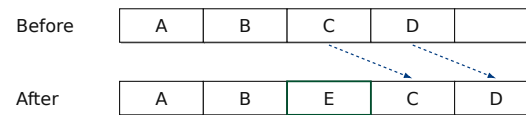
- We want **A → B → E → C → D**. Keep C and D in the playlist.

19/84

With an array, make room for E



- The array has a spare slot. Where do C and D go?



- **C and D move one slot.** A longer suffix means more moves.

20/84

A node stores a song and a link



- A node stores a song ID and the address of the next node.



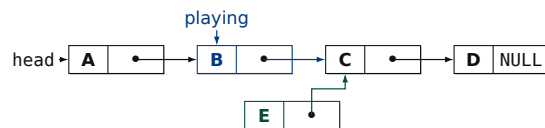
- The dot is a pointer value; the arrow leads to its node. **NULL ends the list.**

21/84

First, let E remember C



- Before changing B, keep its current successor.



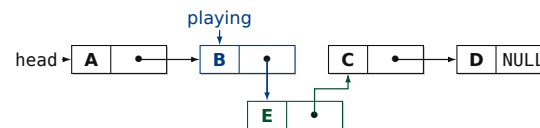
- **E → C.** Following the playlist from *head* still gives A B C D.

22/84

Then, make B lead to E



- Change the next-song link stored in B.



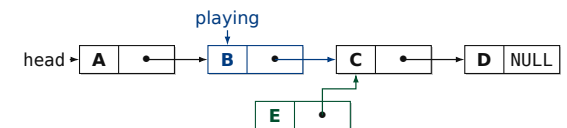
- Follow the arrows: **A → B → E → C → D**. **Two links changed.**

23/84

Cancel the request



- E has not played yet. Make B lead to C again.



- E is **unlinked**. Its storage still needs to be released.

24/84

When links help



- We already know the current song: **B**.
- Adding or removing its successor changes a few links.
- **The other nodes stay in place.**
- Finding song **number 500** requires **following the chain**.
- Arrays give direct indexing and store elements together.

25/84

The same edit with numbers



Insert **101** into `[100, 103, 105, 200]`. What must change?

- **Array with capacity:** move 103, 105 and 200 one slot.
- **List:** find 100, then link **100 → 101 → 103**.
- The list needs an extra pointer per node and sequential search.

26/84

2. Linked-list basics



Turn the drawn playlist into a working C program: **A → B → NULL**.

- Allocate two nodes.
- Link them using *next*.
- Walk the list without changing *head*.

27/84

Can we use the alias here?



```
typedef struct list_elem {
    int value;
    list_elem_t *next;
} list_elem_t;
```

- At *next*, does the compiler already know `list_elem_t`?
- GCC: **unknown type name** '`list_elem_t`'.
- This typedef declares the alias **after the closing brace**.

28/84

A tag inside, an alias afterwards



```
typedef struct list_elem {
    int value;
    struct list_elem *next;
} list_elem_t;
```

- `struct list_elem` is **already known inside the braces**.
- *next* stores a pointer. The node's full size is not needed yet.
- After this declaration, `list_elem_t` names **the same type**.

Now we can use the alias:

```
list_elem_t *head = NULL;
```

29/84

What is in one node?



- A song ID and the address of the next node.



- The next node is a **separate object**, possibly far away in memory.
- *next* stores its address; the nodes need not be adjacent.

30/84

Start with an empty playlist



```
list_elem_t *head = NULL;
```

- `head` is a **local pointer** in `main()`.
- Its value is **`NULL`**: there is no first node yet.
- No `malloc()` call, so **no heap node**.

The diagram illustrates the memory layout of a program. It is divided into two main sections: **Stack (main)** and **Heap**.

- Stack (main):** Located at the top of the memory, starting at address `0xffff`. It contains a `head` pointer and a `NULL` value.
- Heap:** Located at the bottom of the memory, starting at address `0x0000`. It contains vertical ellipses (`:`) indicating memory growth.

31/84

Allocate the first song



```
list_elem_t *node =
    malloc(sizeof(list_elem_t));
if (node == NULL)
    return EXIT_FAILURE;
node->value = 'A';
node->next = NULL;
head = node;
```

- Here: **heap storage** to control lifetime across functions.
- A local node would work for this small example.
- Heap storage also suits potentially large objects.

Stack (main)	
0xffff	
0xffff8	head 0x1000
0xffff0	node 0x1000
	:
Heap	
	:
0x1008	next NULL
0x1000	value 'A'
0x0000	

32/84

Add a second song



```
node = malloc(sizeof(list_elem_t));
if (node == NULL) {
    free(head);
    return EXIT_FAILURE;
}
node->value = 'B';
node->next = NULL;
head->next = node;
```

- A and B are separate allocations.
- A's *next* stores B's address.

Stack (main)		
0xffff		
0xffff8	head	0x1000
0xffff0	node	0x3000
		:
Heap		
0x3008	next	NULL
0x3000	value	'B'
		:
0x1008	next	0x3000
0x1000	value	'A'
0x0000		

33/84

Read the playlist



```
list_elem_t *p = head;
while (p != NULL) {
    printf("%c -> ", p->value);
    p = p->next;
}
puts("NULL");
```

- Only p advances.
- $head$ still reaches the whole list.

Stack (main)	
0xfffff	
0xffff8	head 0x1000
0xffff0	node 0x3000
0xfffe8	p NULL
	:
Heap	
0x3008	next NULL
0x3000	value 'B'
	:
0x1008	next 0x3000
0x1000	value 'A'
0x0000	

34/84

3. Create nodes in a function



Creating each song repeats the same allocation code.
Let `create_element()` do that work for us.

Create a node in a function



What survives when this function returns?

```
list_elem_t *create_element(int value) {
    list_elem_t *node = malloc(sizeof(list_elem_t));
    if (node == NULL) return NULL;
    node->value = value;
    node->next = NULL;
    return node;
}
```

```
In main(): list_elem_t *node = create_element('A');
```

36/84

Université de Rennes
Louis Ledoux

Call `create_element('A')`

```
/* main pauses here */
list_elem_t *node = create_element('A');
```

Stack: local variables

`create_element() (callee)`

value

'A'

main() (caller)

head

NULL

node

waiting

No node allocated yet

- `main()` waits. The called function gets **its own stack frame**.

37/84

Université de Rennes
Louis Ledoux

`malloc()` reserves the node

```
list_elem_t *node = malloc(sizeof(list_elem_t));
if (node == NULL) return NULL;
```

Stack: local variables

`create_element() (callee)`

value

'A'

node

0x1000

main() (caller)

head

NULL

node

waiting

Heap

0x1000

value

?

next

?

- Here `malloc()` returns **0x1000**. ? means uninitialised.

38/84

Université de Rennes
Louis Ledoux

Initialise `value` and `next`

```
node->value = value;
node->next = NULL;
```

Stack: local variables

`create_element() (callee)`

value

'A'

node

0x1000

main() (caller)

head

NULL

node

waiting

Heap

0x1000

value

A

next

NULL

- > follows `node` to write **the heap structure**.

39/84

Université de Rennes
Louis Ledoux

Return the address to `main()`

```
/* inside create_element */
return node;
```

Stack: local variables

`create_element() (callee)`

value

'A'

node

0x1000

main() (caller)

head

NULL

node

0x1000

Heap

0x1000

value

A

next

NULL

- The return value copies **0x1000** into `main()`'s `node`.

40/84

Université de Rennes
Louis Ledoux

The call ends; the node remains

```
/* back in main: the call has completed */
list_elem_t *node = create_element('A');
```

Stack: local variables

`create_element() (callee)`

value

'A'

node

0x1000

main() (caller)

head

NULL

node

0x1000

Heap

0x1000

value

A

next

NULL

- The local pointer is gone; **the heap node stays allocated**.

41/84

Université de Rennes
Louis Ledoux

Make the node the first one

```
/* in main */
head = node;
```

Stack: local variables

`create_element() (callee)`

value

'A'

node

0x1000

main() (caller)

head

0x1000

node

0x1000

Heap

0x1000

value

A

next

NULL

- Two pointers hold **the same address**. There is still one node.

42/84

4. Change the caller's head

Universit  de Rennes

Louis Ledoux

We want `insert_head(...)` to put a new song first.

How can it update `head` in `main()`?

43/84

Can the function change `head`?

Universit  de Rennes

Louis Ledoux

Will the caller's `head` change?

```
int insert_head(list_elem_t *l, int value) {
    list_elem_t *node = create_element(value);
    if (node == NULL) return -1;
    node->next = l;
    l = node;
    return 0;
}
```

Call: `insert_head(head, 'A')`.

44/84

The argument is copied into `l`

Universit  de Rennes

Louis Ledoux

```
insert_head(head, 'A');
/* parameter: list_elem_t *l */
```

Stack: local variables

insert_head() (callee)

l

value

main() (caller)

head

Heap

No node allocated yet

- `l` gets `NULL`, the value of `head`. The variables are separate.

45/84

The function creates a node

Universit  de Rennes

Louis Ledoux

```
list_elem_t *node = create_element(value);
node->next = l;
```

Stack: local variables

insert_head() (callee)

l

value

node

main() (caller)

head

Heap

0x1000

value

next

A

NULL

- `node->next` gets `NULL`. **head is still `NULL`.**

46/84

`l = node` changes only `l`

Universit  de Rennes

Louis Ledoux

```
l = node;
```

Stack: local variables

insert_head() (callee)

l

value

node

main() (caller)

head

Heap

0x1000

value

next

A

NULL

- `l` changes. `head` is a separate variable and stays `NULL`.

47/84

Return loses the node's address

Universit  de Rennes

Louis Ledoux

```
/* inside insert_head */
return 0;
```

Stack: local variables

insert_head(): call finished

main() (caller)

head

Heap

0x1000

value

next

A

NULL

Unreachable

- The local pointers are gone; **the node is unreachable.**

48/84

Universit  de Rennes
Louis Ledoux

Pass the address of head

```
insert_head(&head, 'A');  
/* parameter: list_elem_t **l */
```

Stack: local variables

insert_head() (callee)

l

value

main() (caller)

head at 0x7000

NULL

Heap

No node allocated yet

• Here &head is 0x7000. *l refers to head itself.

Universit  de Rennes
Louis Ledoux

Read the old head through *l

```
list_elem_t *node = create_element(value);  
node->next = *l;
```

Stack: local variables

insert_head() (callee)

l

value

node

main() (caller)

head at 0x7000

NULL

Heap

0x1000

value

next

A

NULL

• Read *l: NULL, the current value stored in head.

Universit  de Rennes
Louis Ledoux

Write head through *l

```
*l = node;
```

Stack: local variables

insert_head() (callee)

l

value

node

main() (caller)

head at 0x7000

0x1000

Heap

0x1000

value

next

A

NULL

• head becomes 0x1000. l still contains &head.

Universit  de Rennes
Louis Ledoux

Back in main(): head is updated

```
/* insert_head has returned */  
printf("head -> %c -> NULL\n", head->value);
```

Stack: local variables

insert_head(): call finished

main() (caller)

head at 0x7000

0x1000

Heap

0x1000

value

next

A

NULL

• C still passes by value. The copied value was &head.

Universit  de Rennes
Louis Ledoux

5. Insert without losing links

A new song must fit before, between, or after the existing songs.
Which links need to change?

Universit  de Rennes
Louis Ledoux

Insert at the head · starting state

head

A

•

B

•

C

•

D

NULL

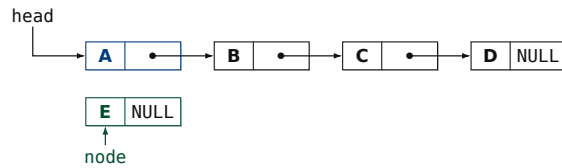
E

NULL

node

• A → B → C → D. A newly allocated node E is ready to link.

Insert at the head · which link matters?



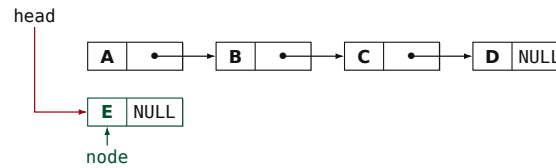
- `head` is our entry point to the existing chain.

55/84

If we change `head` first



What remains reachable after `head = node`?

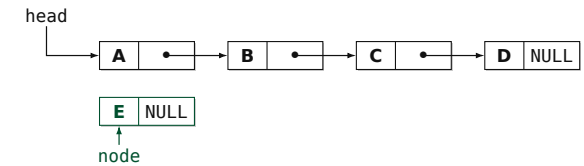


```
head = node;
```

- Only **E** remains reachable; **A-D leak**.

56/84

Back to the initial list



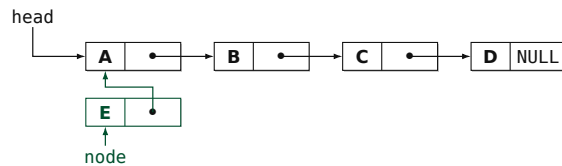
- Reset to the initial state. Preserve the old address before changing `head`.

57/84

Link E to A



Where must E point before we change `head`?



```
node->next = head;
```

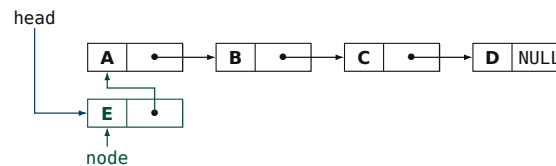
- The old chain is now reachable through **E**.

58/84

Move `head` to E



E already points to A. What must change now?

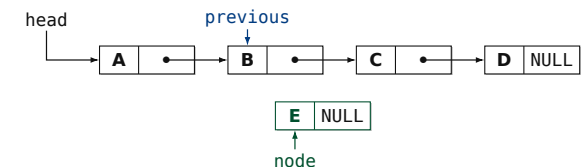


```
head = node;
```

- **E → A → B → C → D**.

59/84

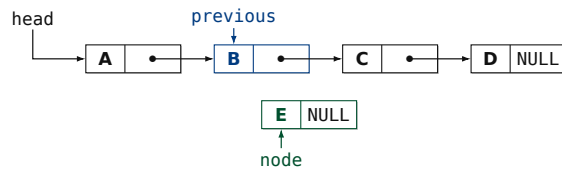
Insert in the middle · starting state



- Insert **E** between **B** and **C**. `previous` points to **B**.

60/84

Insert in the middle · the link to preserve



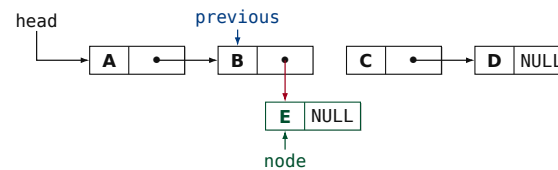
- B's next link is currently our route to C and the rest of the chain.

61/84

If B points to E first



What remains reachable after changing B's link first?

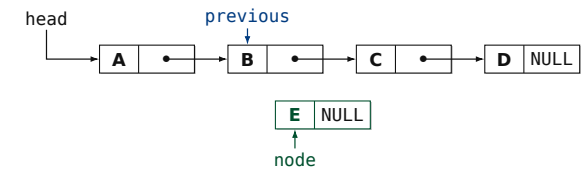


```
previous->next = node;
```

- The list stops at E; **C and D leak**.

62/84

Back to the initial list



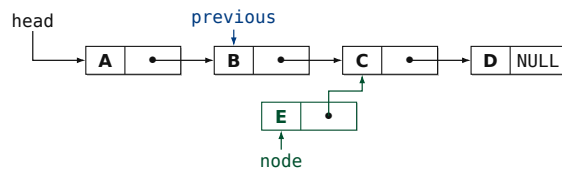
- Reset to A → B → C → D with E still separate.

63/84

Link E to C



Which address should E's *next* receive?



```
node->next = previous->next;
```

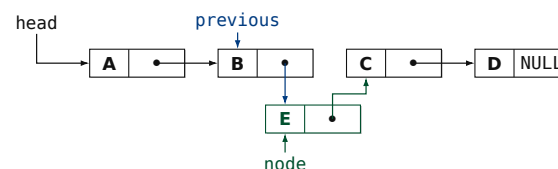
- E points to C; B still points to C.

64/84

Link B to E



E already points to C. Which link finishes the insertion?

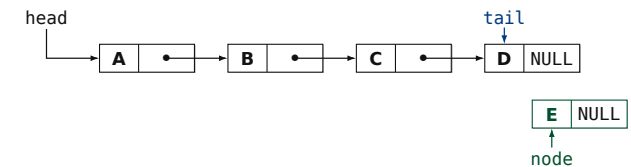


```
previous->next = node;
```

- The list is now **A → B → E → C → D**.

65/84

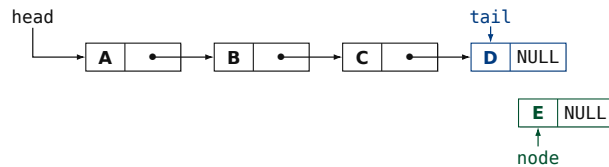
Insert at the tail · starting state



- Append **E** after **D**. *tail* points to D.

66/84

Insert at the tail · find the final link



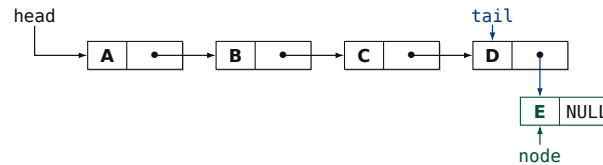
- D's next link currently contains **NULL**.

67/84

Insert at the tail · link D to E



- Which link attaches E to the chain?



```
tail->next = node;
```

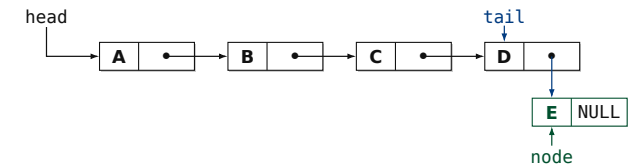
- Connect the old tail to the new node.

68/84

Insert at the tail · terminate the chain



What makes E the last node?



```
node->next = NULL;
```

- The list is now A → B → C → D → E → NULL.

69/84

6. Unlink, then free



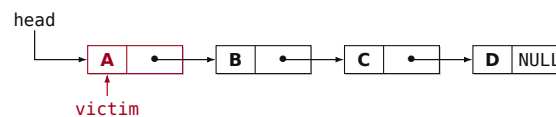
A song is removed from the playlist.
Reconnect the remaining songs and release its memory.

70/84

Delete the head · save the victim



- Which address must we keep before changing *head*?

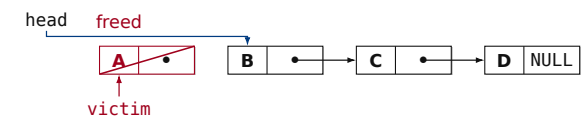


```
list_elem_t *victim = head;
```

- Keep the address of A so it can be freed after unlinking.

71/84

Delete the head · unlink, then release

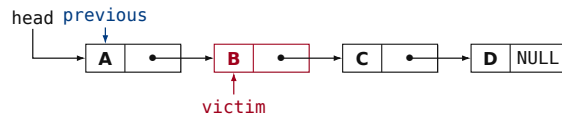
Where should *head* point before we release A?

```
head = victim->next;  
free(victim);
```

- B is now the first node. **Do not access A after free().**

72/84

Delete in the middle · save B



```
list_elem_t *victim = previous->next;
```

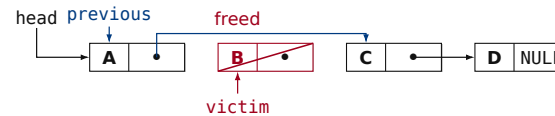
- *previous* points to A. Save the node to be removed.

73/84

Delete in the middle · reconnect A to C



- How do we bypass B before releasing it?

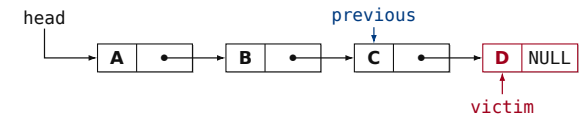


```
previous->next = victim->next;  
free(victim);
```

- **Preserve the successor** before freeing the removed node.

74/84

Delete the tail · find its predecessor



```
list_elem_t *victim = previous->next;
```

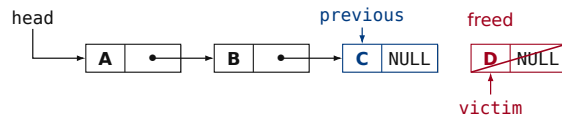
- *previous* points to C; save its successor D.

75/84

Delete the tail · make C the last node



What must C's next link become?



```
previous->next = NULL;  
free(victim);
```

- Set C's next link to **NULL**, then free D.

76/84

7. Doubly linked lists



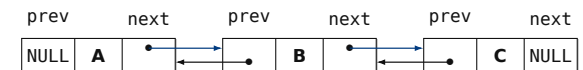
- To delete D, we first had to **find C**.
- D's *next* link cannot take us back to C.
- What if each node **remembered its predecessor**?

77/84

A link in each direction



What extra address should each node store?



- Add *prev*: **reach the predecessor directly** to unlink a known node.
- Also traverse in both directions: our player can offer a **Previous** button.
- **One extra pointer per node**; maintain **both links**.

78/84

Time and storage costs



n nodes; current node known, predecessor not saved.

Operation	Singly linked	Doubly linked
Find its predecessor	$O(n)$	$O(1)$
Unlink this node	$O(n)$	$O(1)$
Find node number k	$O(n)$ worst case	$O(n)$ worst case
Total node storage	$\Theta(n)$	$\Theta(n)$

- Singly linked: start at *head*, follow *next* to the predecessor.
- Doubly linked: read *prev* directly; **one extra pointer per node.**

79/84

8. Questions



Choose **A, B, C or D**, then explain why.

80/84

Linked lists · q1: changing the head



```
void prepend(list_elem_t *head,
             list_elem_t *node) {
    node->next = head;
    head = node;
}
```

head points to A; *node* points to a separate node E.

After `prepend(head, node)`,
what happens to the caller's
head?

- A. It points to E.
- B. **It still points to A.**
- C. It becomes **NULL**.
- D. It points to a copy of A.

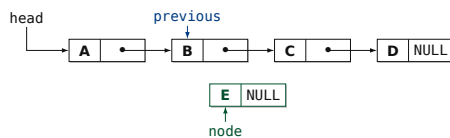
- The function changes **its local pointer copy**.
- E points to A, but the caller's *head* is unchanged.

81/84

Linked lists · q2: insertion order



Insert E between B and C. Which order works?



```
/* X */ previous->next = node;  
/* Y */ node->next = previous->next;
```

- B. **Y, then X**

- **E remembers C** before B points to E.

82/84

Linked lists · q3: releasing a node



victim points to the first node of a nonempty list.

```
free(victim);
head = victim->next;
```

What is the problem?

- A. The whole list is freed.
- B. *head* must become **NULL**.
- C. **We read a freed node.**
- D. There is no problem.

- Read `victim->next` before `free()`.
- The invalid read has undefined behaviour; a crash is not guaranteed.

83/84

Linked lists · q4: time and storage



A doubly linked list has n nodes; the current node is known.

Reading *prev*: what time cost and **total** node storage?

- A. $O(1)$ time; $\Theta(n)$ storage
- B. $O(n)$ time; $\Theta(n)$ storage
- C. $O(1)$ time; $\Theta(1)$ storage
- D. $O(n)$ time; $\Theta(n^2)$ storage

- **Direct access** to the predecessor; no traversal.
- **One extra pointer per node**; total storage stays linear.

84/84